

课程笔记

22 - 输入输出设备和驱动

2026 南京大学操作系统原理

lecture-to-notes

2026



视频作者/频道：南京大学操作系统原理（蒋炎岩）

发布日期：2026

视频时长：01:26:56

视频链接：未填写

目录

1 从系统调用对象走向 I/O 设备	2
1.1 文件描述符是 OS 对象的把柄	2
1.2 本章小结	3
2 Canonical Device Model: 状态、命令、数据	3
2.1 端口 I/O 与内存映射 I/O	3
2.2 中断: 从轮询到异步通知	4
2.3 本章小结	4
3 从 LED 到经典 PC 外设	4
3.1 UART: 一个字符终端的硬件根基	5
3.2 PS/2 键盘: 输入设备也是寄存器协议	5
3.3 ATA 磁盘: 块设备的 programmed I/O	6
3.4 本章小结	7
4 设备可以是协议解释器	7
4.1 打印机: 从机械自由度到页面描述语言	7
4.2 UVC 摄像头: 标准化让设备免驱	8
4.3 GPU/NPU: 设备也可以是协处理器	9
4.4 本章小结	9
5 总线、DMA 与共享内存	9
5.1 总线是设备管理器	10
5.2 DMA: 让搬运数据的工作离开 CPU	10
5.3 共享内存、缓存一致性与 CXL	11
5.4 本章小结	11
6 操作系统如何抽象设备	11
6.1 struct file_operations: 把设备接入文件系统	12
6.2 课堂演示: 一个字符设备驱动	12
6.3 ioctl: 干净抽象下面的控制冰山	13
6.4 UVC、GPU 与 KVM: 同一个控制平面	13
6.5 本章小结	14
7 总结与延伸	14
7.1 讲者的收束	14
7.2 概念压缩	15
7.3 学习建议	15
7.4 本章小结	15

1 从系统调用对象走向 I/O 设备

本讲是从 Virtualization 与 Concurrency 进入 Persistence 之前的一座桥。前面课程已经把 Unix 世界里的进程、地址空间、线程、文件描述符和系统调用串起来：用户程序通过 open、read、write、mmap、fork、exec 等系统调用访问操作系统对象。现在要打开另一个黑盒：这些对象背后的物理设备到底是什么，设备如何被抽象成文件，又如何被设备驱动程序接入内核。

本讲的中心命题

I/O 设备是计算机内部状态机与物理世界之间的边界。应用程序看到的是文件描述符、路径和系统调用；CPU 看到的是寄存器、内存映射、中断和总线事务；设备驱动程序负责把两者翻译到同一个可维护的操作系统对象模型中。

“输入/输出”总是站在 CPU 的角度命名：外部世界送入机器的数据是输入，机器向外部世界施加的影响是输出。我们日常所见的计算机其实几乎全是 I/O 设备：屏幕、键盘、鼠标、摄像头、打印机、磁盘、网卡。CPU 和内存本身不可直接改变现实世界，只有经由这些设备，机器内部的 0/1 状态才能变成光、电、声音、图像、运动或持久化数据。

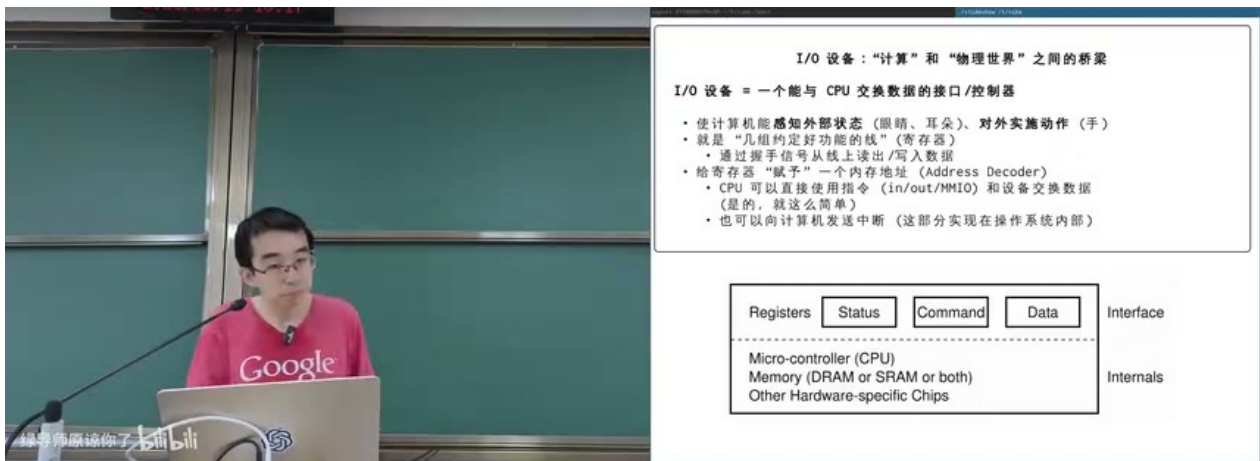


图 1: Canonical Device Model: 设备以少量寄存器暴露给 CPU，内部复杂性被接口隐藏。¹

1.1 文件描述符是 OS 对象的把柄

Unix/Linux 设计中，文件描述符是指向操作系统对象的 handle。文件可以是普通文件、目录、管道、终端、磁盘、摄像头、GPU 控制节点，也可以是 /proc 里的伪文件。这个统一性带来一个强大的工程风格：应用程序尽可能使用同一组 API 访问不同对象；差异留给内核、驱动和设备协议处理。

这也解释了为什么 I/O 设备会出现在文件系统中。例如：

- /dev/tty 抽象出字符终端；

¹ 视频画面时间区间：00:03:40-00:03:45。

- `/dev/sda`、`/dev/nvme0n1`、`/dev/mmcblk0` 抽象出块设备；
- `/dev/video0` 抽象出 UVC 摄像头；
- `/dev/kvm` 抽象出内核中的虚拟机子系统；
- `/sys/class/leds/.../brightness` 抽象出一个可写的 LED 亮度控制接口。

1.2 本章小结

操作系统课前半部分讲“CPU 内部和进程世界”，本讲开始把视角推到机器外壳之外。I/O 设备不是外围装饰，而是计算机与现实世界交互的全部入口和出口。文件描述符统一了应用程序视角；设备驱动程序把物理设备接入这个统一视角。

2 Canonical Device Model: 状态、命令、数据

从 CPU 看，一个设备再复杂，也可以被压缩成一组可读写的寄存器。最典型的抽象是三个寄存器类别：

- **Status**: 设备是否 ready、是否 busy、是否出错、是否有数据到达；
- **Command**: CPU 对设备发出的控制命令，例如开始、停止、复位、配置模式；
- **Data**: CPU 与设备交换的数据本体，例如字符、扇区内容、图像帧、命令队列。

设备内部可以是机械结构、微控制器、DMA 引擎、固件、缓存、片上内存、甚至另一台小计算机；但 CPU 不需要直接知道这些细节。CPU 只需要能把值写到某个寄存器，或从某个寄存器读出值。

2.1 端口 I/O 与内存映射 I/O

设备寄存器如何被 CPU 访问，有两类经典路径。

两种常见的寄存器寻址方式

Port-mapped I/O 把设备寄存器放在独立的 I/O 地址空间，x86 的 `in/out` 指令就是典型例子。

Memory-mapped I/O 把设备寄存器映射到物理地址空间，CPU 用普通 `load/store` 访问特殊地址，硬件在总线层面把访问转发到设备。

本质上，这两种方式都在做同一件事：当 CPU 发出一次读写时，硬件多路选择器判断这次访问应该落到内存，还是落到某个设备寄存器。于是设备的“接口”就被放进了 CPU 可见的地址体系。

2.2 中断：从轮询到异步通知

只有寄存器还不够。若 CPU 不停轮询 `status`，会浪费大量时间。于是设备还可以通过中断线通知 CPU：数据到了、传输完成了、设备出错了。操作系统收到中断后进入中断处理程序，再唤醒等待该事件的内核线程或用户进程。

可以把中断理解为内核态的条件变量通知：等待设备的线程睡眠；设备触发中断；中断处理程序记录事件并唤醒等待者；调度器稍后让相应线程继续执行。

不要把中断想成“设备自己运行用户代码”

中断只是在 CPU 控制流中强制跳转到预先注册的内核处理例程。设备不会直接进入用户进程；真正的隔离、唤醒、复制和权限检查仍由操作系统完成。

2.3 本章小结

I/O 设备的最小抽象是“寄存器 + 可选中断”。状态寄存器回答“能不能做”，命令寄存器表达“要做什么”，数据寄存器承载“交换什么”。端口 I/O、内存映射 I/O、中断、轮询都是围绕这三个问题展开的实现机制。

3 从 LED 到经典 PC 外设

讲清楚 I/O 设备最好的方式，是从一个 LED 开始。LED 只需要一个可写寄存器：写 1 亮，写 0 灭。它没有复杂协议，不需要 DMA，不一定需要中断，却已经完整体现了“CPU 状态改变物理世界”的本质。

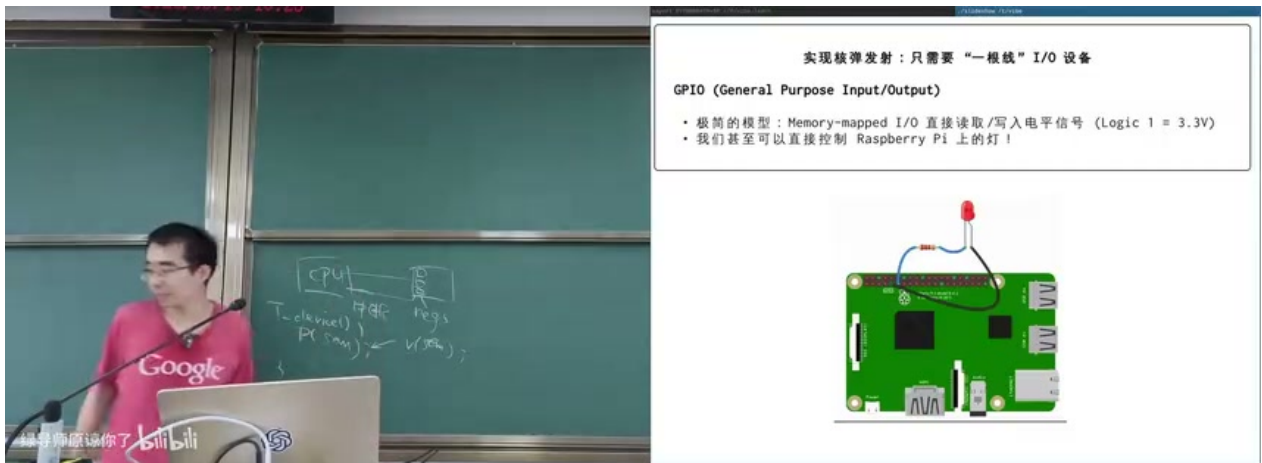


图 2: GPIO/LED 是最小 I/O 设备：一个寄存器足以改变外部电平和发光状态。²

在 Linux 用户态，树莓派 LED 可以通过 `sysfs` 暴露成一个可写文件，例如名为 `brightness` 的亮度节点。用户程序执行 `write(fd, "1", 1)`，表面上是在写文件；底层最终会变成驱动写某个内存映射寄存器。

²视频画面时间区间：00:09:30–00:09:35。

```

1 int fd = open("/sys/class/leds/power/brightness", O_WRONLY);
2 write(fd, "1", 1); // LED on
3 write(fd, "0", 1); // LED off
4 close(fd);

```

Listing 1: 用户态通过文件接口控制 LED 的典型结构

3.1 UART: 一个字符终端的硬件根基

串口是从 LED 走向“可用计算机”的第一步。UART 把字节按约定的时序发送到另一端；只要有串口，机器就可以有终端、日志、调试口、引导控制台。

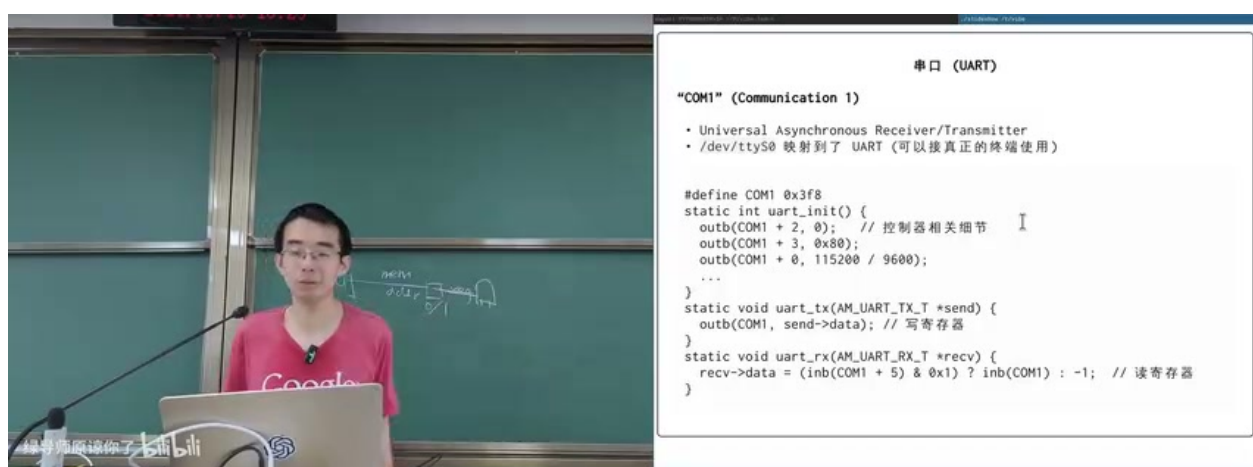


图 3: COM1/UART 示例：通过端口地址配置控制寄存器并读写数据寄存器。³

x86 上 COM1 的常见基地址是 0x3f8。写字符时，CPU 实际写的是发送数据寄存器；读字符时，CPU 先看状态寄存器，再从数据寄存器取字节。这里已经能看到 status/command/data 三分法的影子。

3.2 PS/2 键盘：输入设备也是寄存器协议

早期键盘控制器通过 data、clock、VCC、GND 等物理线传输信号，并映射到固定 I/O 端口。CPU 可从 data 端口读扫描码，也可以向 command/status 端口写命令配置键盘行为，例如 LED 灯、按键重复延迟。

³视频画面时间区间：00:13:40–00:13:45。

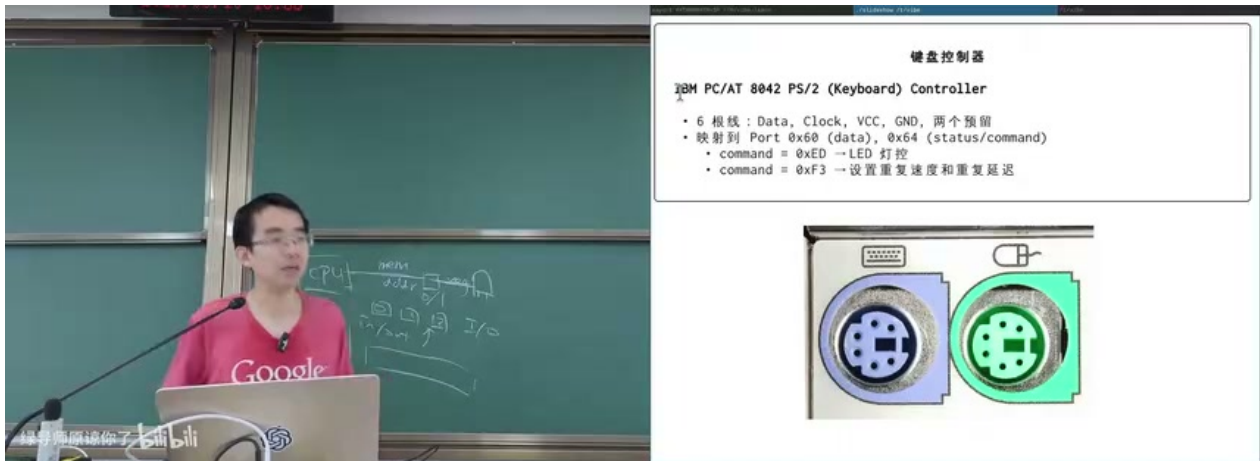


图 4: PS/2 键盘控制器: 端口 0x60/0x64 分别承担数据与状态/命令角色。⁴

这里有一个历史转变: 早期 CPU 慢, 很多重复按键等行为放在硬件控制器中; 今天更多逻辑转移到操作系统和用户态输入栈里。硬件只报告“按下/松开”, 软件决定字符生成、重复延迟、快捷键和输入法。

3.3 ATA 磁盘: 块设备的 programmed I/O

磁盘控制器也可以被写成寄存器协议: 等待 ready, 写入要读取的扇区号、扇区数、命令, 再从数据端口连续读出扇区内容。

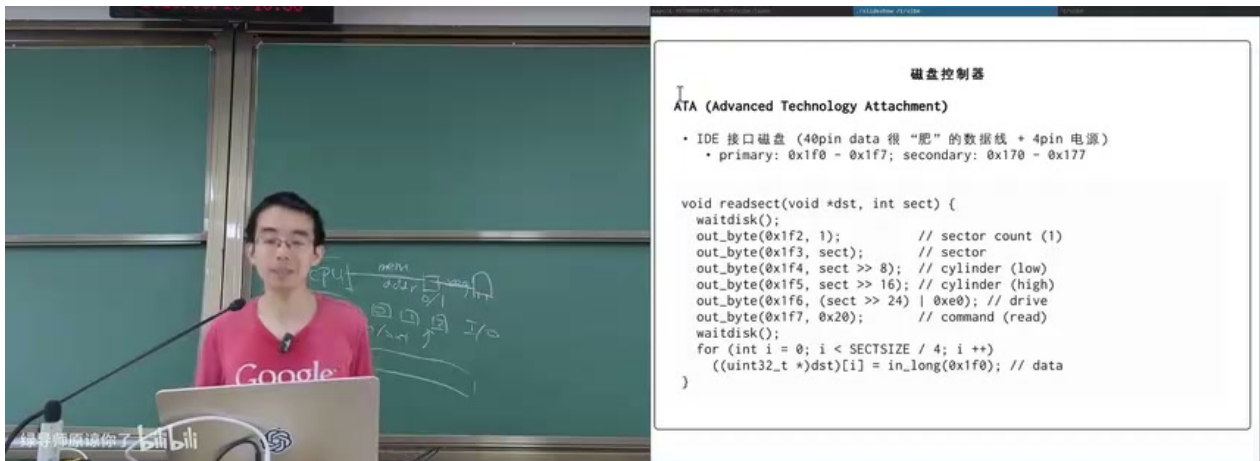


图 5: ATA programmed I/O: CPU 通过端口寄存器指定扇区并循环读取数据。⁵

这种方式直观, 但会让 CPU 被大块数据搬运占住。如果读取 1MB 数据, CPU 需要在循环中反复从端口取值。后面 DMA 和总线的设计, 正是为了把这种搬运工作从主 CPU 身上卸下来。

⁴视频画面时间区间: 00:17:00–00:17:05。

⁵视频画面时间区间: 00:19:35–00:19:40。

3.4 本章小结

LED、UART、PS/2 键盘、ATA 磁盘外形完全不同，但 CPU 看到的仍然是寄存器和协议。设备越复杂，寄存器协议越多，状态机越复杂；但抽象边界仍是“读状态、写命令、搬数据”。

4 设备可以是协议解释器

有些设备不是简单地搬字节，而是在执行一门“语言”。CPU 送入的不是单个值，而是一段程序或命令流；设备内部的小控制器解释这段命令流，产生物理效果。

4.1 打印机：从机械自由度到页面描述语言

早期绘图仪式打印机有几个基本自由度：笔抬起/落下、X 轴移动、Y 轴移动。只要能组合这些动作，就能画线；只要能画线和字符，就能打印复杂图形。

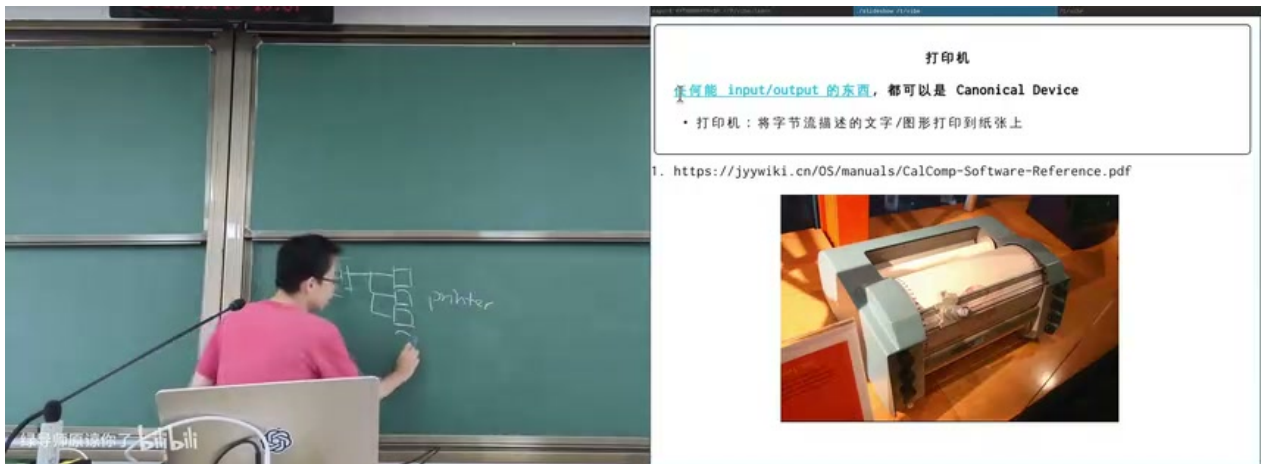


图 6: 早期绘图仪/打印机：机械动作被组织成可发送给设备的控制序列。⁶

随着打印机内部处理能力增强，命令流也升级为 PCL、PostScript、PDF 等页面描述语言。应用程序或排版系统把高层文档编译为设备能理解的程序，打印机解释程序并产生纸面输出。

⁶视频画面时间区间：00:21:10–00:21:15。

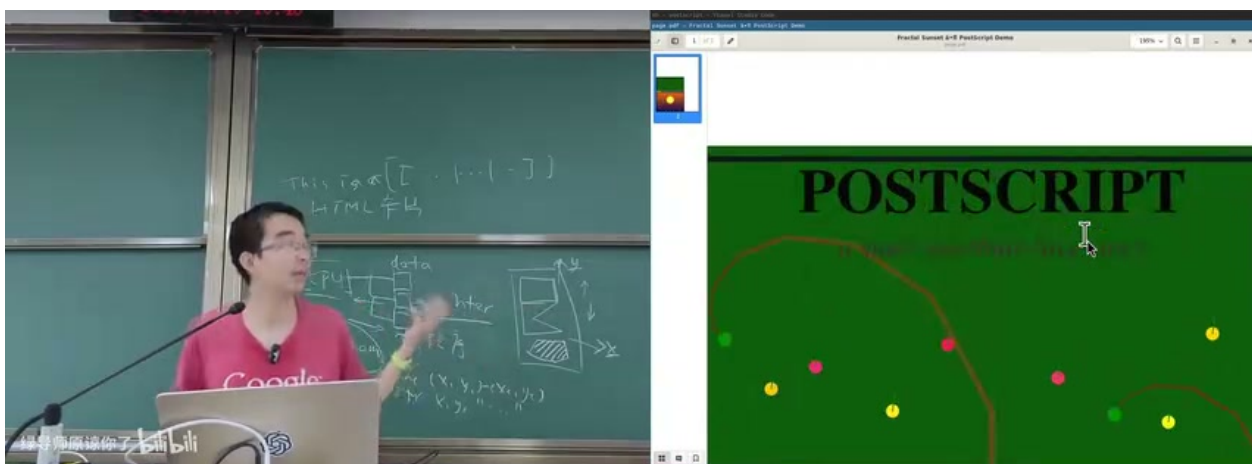


图 7: PostScript 演示：文本、路径、颜色和图形作为程序被解释成页面输出。⁷

打印机为何像一台小计算机

现代打印机不仅有机机械部件，还有缓冲区、解释器、字库、图像处理管线和网络协议栈。CPU 发送给它的常常不是“把这个点打黑”，而是“解释这段页面描述程序”。因此设备协议越高级，驱动与固件承担的语义就越多。

4.2 UVC 摄像头：标准化让设备免驱

USB 摄像头通常遵循 UVC (USB Video Class) 标准。它们和主机协商图像格式、分辨率、帧率、压缩方式，然后逐帧传输图像数据。应用程序看到的是 `/dev/video0`；驱动和 USB 子系统负责把查询能力、设置格式、缓冲区映射、启动流、等待帧等操作翻译成 UVC 协议事务。

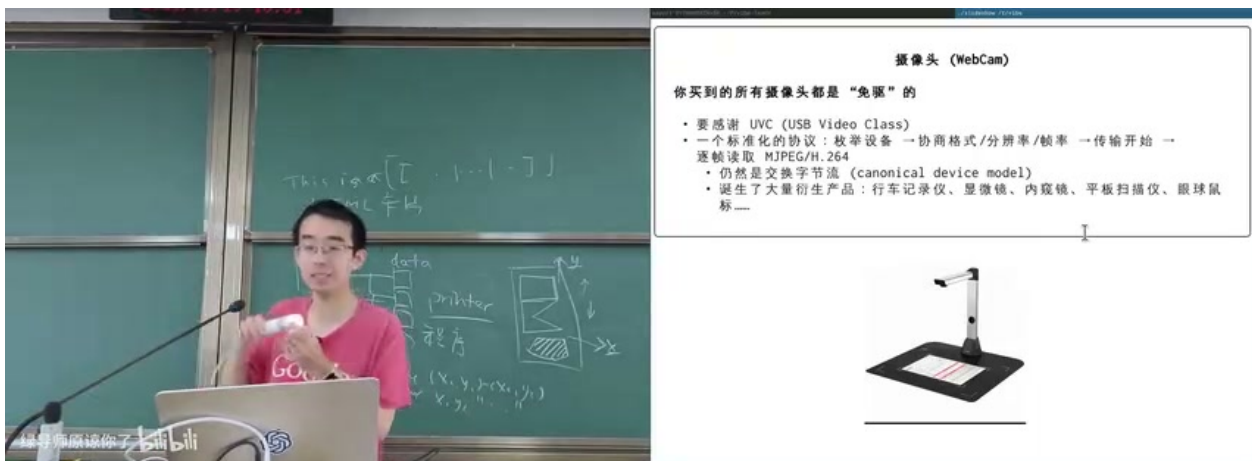


图 8: UVC 摄像头：一类设备共享同一协议，应用程序通过标准接口抓取帧。⁸

UVC 的意义不只是“能读图像”。它展示了一种生态模式：设备厂商实现标准协议，操

⁷视频画面时间区间：00:30:20–00:30:25。

⁸视频画面时间区间：00:32:25–00:32:30。

作系统提供通用驱动，应用程序使用统一 API。这样内窥镜、显微镜、平板扫描仪、行车记录仪等设备都可以复用同一软件栈。

4.3 GPU/NPU：设备也可以是协处理器

GPU 和 NPU 更进一步：它们不是简单解释命令流，而是执行自己的程序。GPU 有显存、执行单元、调度器、DMA 引擎和大量硬件状态。CPU 要把数据、贴图、kernel、命令队列准备好，再通知 GPU 启动执行。

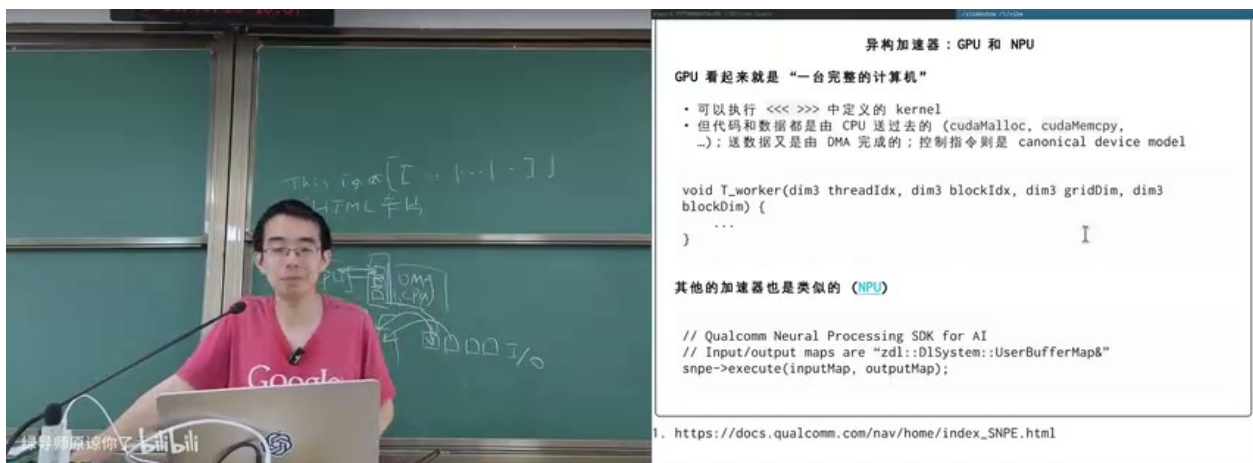


图 9: GPU/NPU 作为异构加速器：CPU 提交代码、数据和启动命令，设备侧执行并返回结果。⁹

这说明“设备”并不等于“被动外设”。现代设备可能有自己的处理器和内存体系，只是它仍通过一组主机可见接口与 CPU 协作。

4.4 本章小结

当设备内部拥有控制器和固件时，CPU 发送的数据就可以升级为命令语言、页面描述、流媒体协商、GPU kernel 或任务队列。设备本质仍是状态机，但这个状态机可以很强大，甚至接近另一台计算机。

5 总线、DMA 与共享内存

早期 PC 可以把端口号写死：COM1 是 0x3f8，PS/2 是 0x60/0x64。但现代机器有大量可热插拔、可组合、可枚举的设备，固定端口模型无法扩展。总线由此成为设备生态的基础设施。

⁹视频画面时间区间：00:38:30–00:38:35。

5.1 总线是设备管理器

总线可以被理解为“管理设备的 I/O 设备”。CPU 与总线通信，总线发现设备、分配资源、转发访问、传递中断、支持 DMA。PCIe、USB 都是这种思想的代表。

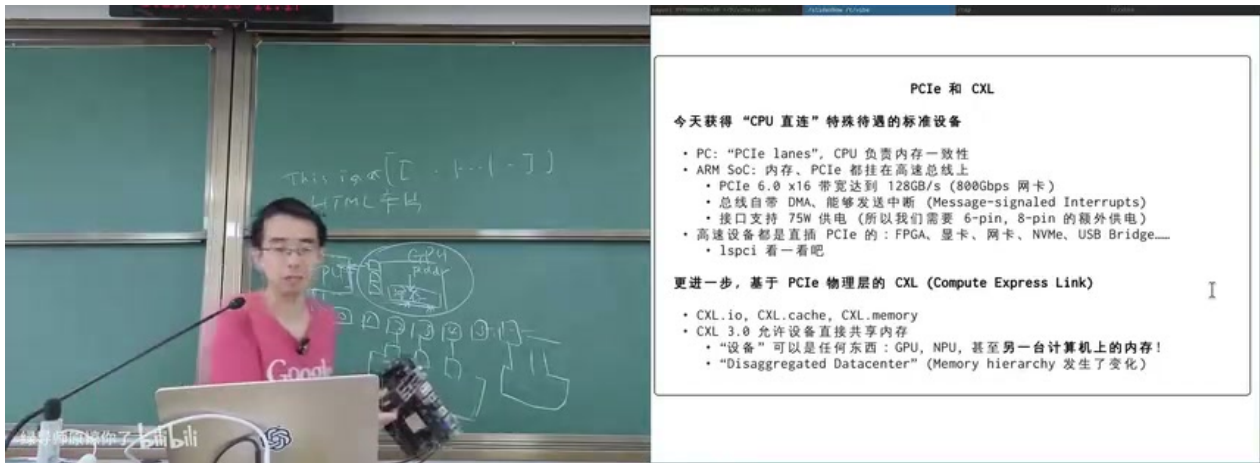


图 10: PCIe/CXL: 现代高速设备通过标准总线获得地址空间、带宽、中断和 DMA 能力。

10

操作系统可以扫描 PCI/USB 拓扑, 得到 vendor/device/class 信息, 选择合适驱动, 并把设备注册到内核对象系统中。热插拔时, 设备变化会通过中断或事件通知内核。历史上即插即用的崩溃案例提醒我们: 驱动运行在内核里, 自动加载和设备枚举一旦有 bug, 影响的是整个系统。

5.2 DMA: 让搬运数据的工作离开 CPU

如果所有磁盘、网卡、摄像头、GPU 数据都靠 CPU 循环读写端口, 系统会被 I/O 搬运拖垮。DMA (Direct Memory Access) 把搬运工作交给专门的控制器: CPU 配置源地址、目的地址、长度和方向, DMA 引擎执行复制, 完成后再通过中断通知。

$$\text{I/O latency} = T_{\text{setup}} + T_{\text{transfer}} + T_{\text{completion}}$$

其中:

- T_{setup} : CPU 配置设备、DMA 描述符和缓冲区的开销;
- T_{transfer} : 总线和设备真正搬运数据的时间;
- $T_{\text{completion}}$: 中断、唤醒、缓存一致性处理和返回用户态的开销。

DMA 优化的不是所有时间, 而是把大块 T_{transfer} 从 CPU 执行流中移走。CPU 可以继续运行其他线程, 系统吞吐量因此提高。

¹⁰ 视频画面时间区间: 00:47:40–00:47:45。

5.3 共享内存、缓存一致性与 CXL

当设备能直接访问内存，就会引入新的系统问题：CPU cache、内存、设备 DMA 写入之间如何保持一致？PCIe 把高速链路和内存体系紧密结合；CXL 进一步把 cache 和 memory 语义带进总线协议，支持本地或远端设备以内存方式被访问。

共享内存不是免费午餐

一旦设备能读写内存，操作系统必须管理权限、地址翻译、IOMMU、缓存一致性和缓冲区生命周期。DMA 提高吞吐，也扩大了错误和攻击面：错误地址可能破坏内核内存，错误同步可能读到旧数据。

5.4 本章小结

总线解决“设备如何被发现和接入”，DMA 解决“大量数据如何不占用 CPU 搬运”，共享内存和 CXL 则把设备推向更深的内存体系协作。现代 I/O 不只是端口读写，而是地址空间、协议、缓存一致性和驱动生态的共同设计。

6 操作系统如何抽象设备

CPU 可以直接访问寄存器，但用户程序不应该这样做。设备是共享资源；多个进程同时写打印机、GPU 或终端会产生竞争。设备还需要权限控制、错误恢复、排队、公平性、缓冲、格式转换和安全检查。这些都属于操作系统职责。

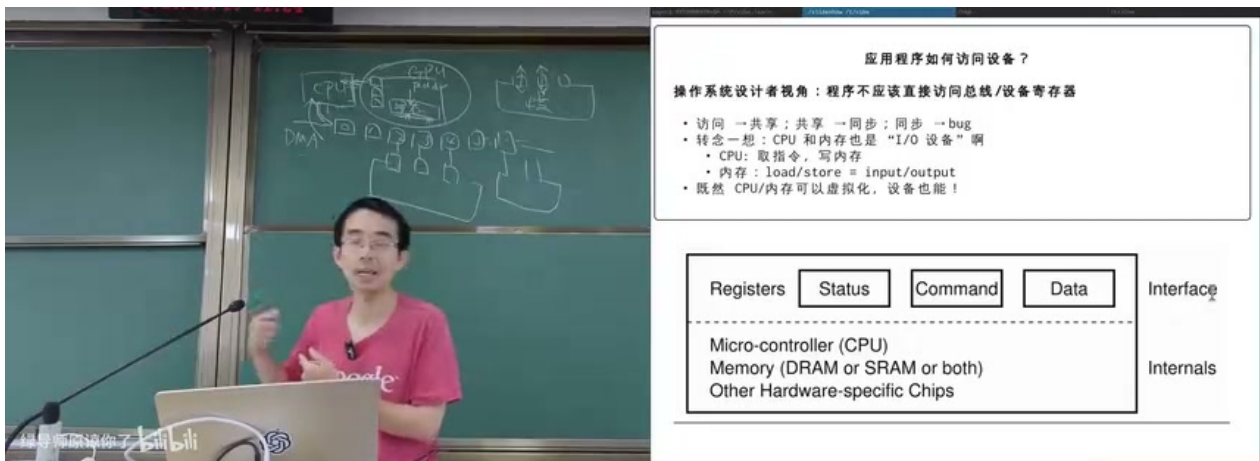


图 11：从 OS 设计者视角看设备：应用程序不应直接访问寄存器，而应通过内核对象和驱动访问设备。¹¹

¹¹ 视频画面时间区间：00:54:05–00:54:10。

6.1 struct file_operations: 把设备接入文件系统

Linux 中, 字符设备、块设备和许多伪设备都通过文件操作表接入 VFS。驱动实现 read、write、mmap、poll、ioctl 等回调; 应用程序对文件描述符发起系统调用; 内核根据打开文件对应的对象找到驱动回调。

```
1 static const struct file_operations dev_fops = {
2     .owner          = THIS_MODULE,
3     .read           = mydev_read,
4     .write          = mydev_write,
5     .mmap           = mydev_mmap,
6     .unlocked_ioctl = mydev_ioctl,
7 };
```

Listing 2: 设备驱动把文件操作翻译成设备协议的基本形态

/dev/null 是最小例子: 读永远返回 EOF, 写直接报告成功但丢弃数据。真实设备驱动则会等待 status、写 command、搬运 data、处理中断, 必要时用 DMA 和队列管理并发请求。

6.2 课堂演示: 一个字符设备驱动

讲者现场演示了一个字符设备驱动: 加载模块后 /dev 中出现设备文件; cat 触发驱动的 read; 向设备写入错误或正确字符串会在内核日志中产生不同结果。这个演示的重点不是具体功能, 而是系统调用如何被分发到驱动回调。

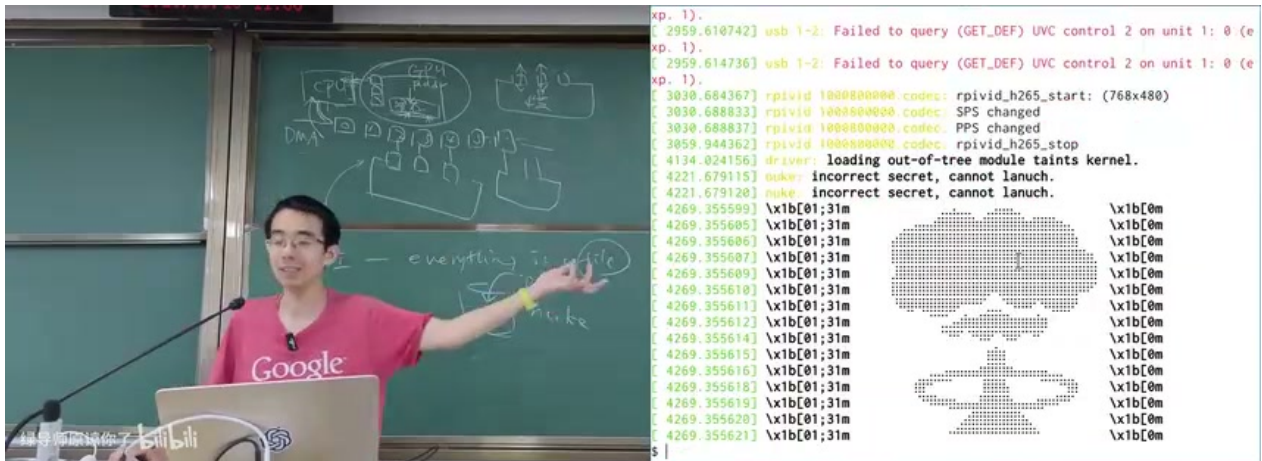


图 12: 驱动演示: 用户态命令经由设备文件触发内核驱动逻辑, 并在日志/终端中观察结果。¹²

¹²视频画面时间区间: 01:05:30–01:05:35。

驱动程序的本质

设备驱动是一段普通但高权限的内核代码。它把稳定的系统调用接口翻译成特定硬件的寄存器读写、DMA 描述符、等待队列、中断处理和错误恢复逻辑。

6.3 ioctl：干净抽象下面的控制冰山

`read/write` 很适合数据流或字节序列：终端、管道、普通文件、简单字符设备、块设备扇区。但许多设备还需要控制平面：打印机清洁、卡纸处理、装订模式；终端窗口大小和模式；键盘灯效；磁盘健康状态；摄像头格式协商；GPU 命令提交；KVM 虚拟机创建。

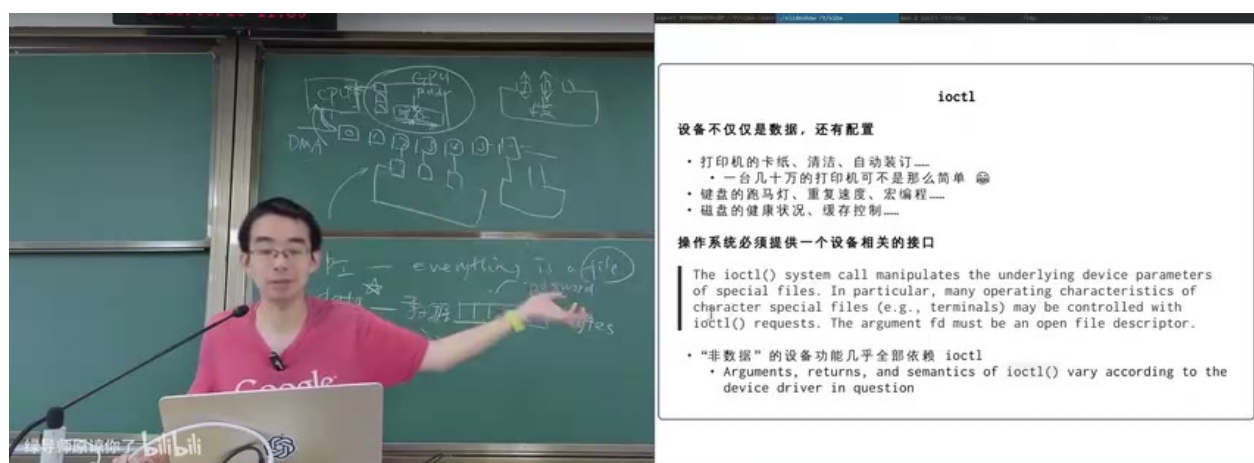


图 13: `ioctl` 承担设备相关控制：参数、返回值和语义由具体驱动定义。¹³

`ioctl(fd, request, arg)` 的特点是：`request` 编码一个设备相关命令，`arg` 往往是指向结构体的指针，语义由应用程序和驱动共同约定。于是 Unix “everything is a file” 的表层很干净，但大量隐藏规格沉入 `ioctl`、`/proc`、`/sys` 和设备私有协议中。

ioctl 的工程代价

`ioctl` 强大，因为它允许任意设备扩展控制语义；也危险，因为它绕开了统一文件 API 的简单模型。兼容 Linux 不只是实现几百个系统调用，还要模拟成千上万个设备私有控制命令和伪文件行为。

这也解释了 WSL 早期 `syscall` 翻译方案的局限：Linux 世界的复杂性不只在 `syscall` 表里，还在 `ioctl`、`procfs`、`sysfs`、驱动行为和大量事实标准中。最终用真正 Linux 内核运行在虚拟机里，反而更容易获得完整兼容。

6.4 UVC、GPU 与 KVM：同一个控制平面

UVC 摄像头抓一帧图像时，应用程序通常不是简单 `read`。它会查询能力、设置格式、申请缓冲区、`mmap` 缓冲区、排队缓冲、启动流、等待一帧、取回缓冲；这些大量由 `ioctl` 完成。

¹³ 视频画面时间区间：01:08:20–01:08:25。

GPU 也类似：用户态库如 Mesa/libdrm 封装 `ioctl`，提交命令缓冲和内存对象；KVM 更极端，一个虚拟机的创建、内存注册、VCPU 创建、寄存器设置和运行都可以通过 `/dev/kvm` 上的一系列 `ioctl` 完成。

6.5 本章小结

操作系统不让应用程序直接操作设备寄存器，而是把设备变成内核对象和文件描述符。驱动实现文件操作，把系统调用翻译成设备协议。`read/write` 是数据平面，`ioctl` 是控制平面；真实世界的复杂性大多沉在控制平面里。

7 总结与延伸

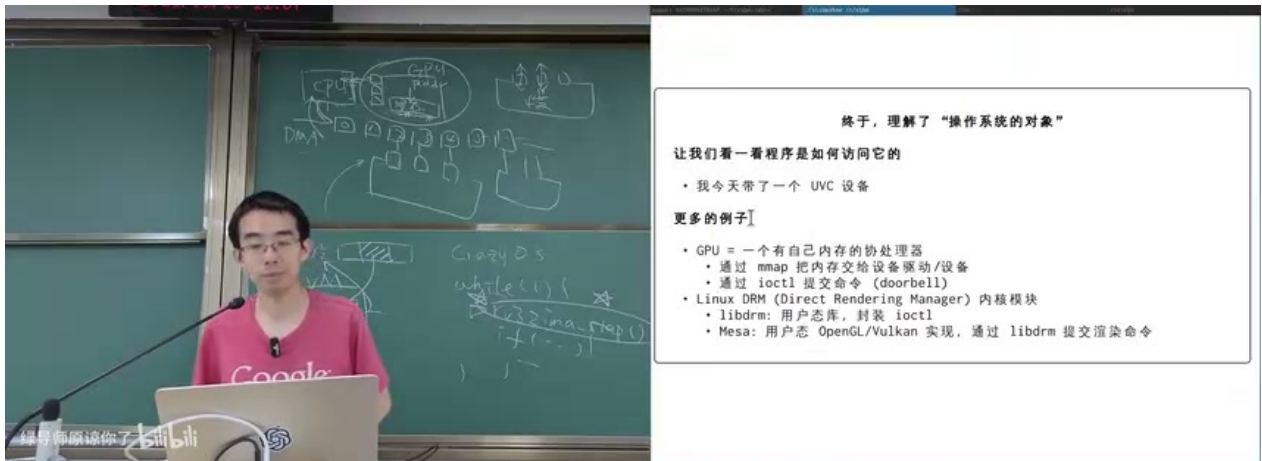


图 14: 结尾总结：从 UVC、GPU 到 KVM，应用程序访问的是操作系统对象，最终落到设备协议和驱动。¹⁴

7.1 讲者的收束

本讲最后回到“操作系统里的对象”。程序发出的每一次 `read/write/ioctl/mmap`，在内核中兜一圈后，最终会变成总线上某个设备可理解的读写、命令、缓冲区、DMA 或中断。无论是 LED、打印机、摄像头、GPU，还是 `/dev/kvm`，核心机制都可以从最简单的“写一个寄存器改变电平”开始理解。

讲者也特别强调实践方法：面对不熟悉的设备，不要停留在“驱动很神秘”。可以用 `strace`、系统调用序列、`lsusb -t`、`lspci`、`/proc`、`/sys` 等工具观察真实交互，再让工具或 AI 帮助解释这些调用背后的设备协议。

¹⁴视频画面时间区间：01:24:20–01:24:25。

7.2 概念压缩

一张压缩图

应用程序 → 文件描述符 → VFS/内核对象 → 驱动回调 → 寄存器/命令队列/DMA
→ 设备内部状态机 → 物理世界。

本讲可以压缩成四个层次：

- **硬件层**：设备以寄存器、内存映射、中断、DMA、总线事务暴露能力；
- **协议层**：UART、PS/2、ATA、UVC、PostScript、PCIe、CXL 等定义数据和命令格式；
- **内核层**：驱动实现设备协议，提供 `file_operations`、缓冲、同步、权限和错误处理；
- **应用层**：程序使用路径、文件描述符、系统调用和用户态库访问对象。

7.3 学习建议

- 从 `/dev/null`、`/dev/zero`、`/dev/random` 这些伪设备开始，理解文件操作如何被驱动实现；
- 用 `strace` 观察 `cat /dev/null`、摄像头抓帧、终端窗口 `resize`、GPU 程序启动时的系统调用；
- 用 `lsusb -t` 与 `lspci -vv` 观察设备拓扑、驱动绑定和带宽信息；
- 阅读一个最小字符设备驱动，重点看初始化/清理、设备号注册和 `file_operations`；
- 继续追踪 `copy_to_user`、`copy_from_user` 与 `ioctl` 如何形成用户态和驱动之间的边界；
- 把“设备”理解为可编程状态机，而不是一块神秘硬件。

7.4 本章小结

I/O 设备和驱动把操作系统课程前面的对象模型、并发同步、地址空间、权限隔离和系统调用全部连接起来。Persistence 部分将继续沿着这条线进入磁盘和文件系统：磁盘首先是设备，其次才是块设备，最后才在操作系统抽象中变成目录、文件、缓存和一致性协议。